

Lecture 14. SAGA and Transaction Processing



Lee Youngkon



Saga Pattern in Microservices (MSA)

- ✓ A Saga is a design pattern for managing distributed transactions across multiple microservices.
- ✓ Instead of using a single global transaction, a saga coordinates a sequence of local transactions.
- ✓ If one transaction fails, compensating transactions are executed to undo the previous successful operations.

Saga Pattern in Microservices (MSA)

1. Successful Flow



2. Failure & Compensation Flow (e.g., Payment Fails)



Key Concepts of Saga

- ✓ Distributed Transaction
 - Each microservice manages its own database.
 - No shared database or global ACID transaction.
- ✓ Local Transaction
 - Each service completes its own database transaction independently.
- ✓ Event-Driven Coordination
 - Services communicate using events or messages.
- ✓ Compensating Transaction
 - Reverses previously completed actions when a failure occurs.
- ✓ Eventual Consistency
 - Data becomes consistent over time instead of immediately.

Benefits and Challenges



Benefits



Scalability

Services can scale independently.



Loose Coupling

Services are independent and communicate via events.



High Availability

No single point of failure.



Fault Tolerance

Compensating transactions handle failures.



Better Performance

Asynchronous processing improves responsiveness.



Challenges



Complexity

Harder to design and implement than local transactions.



Compensating Transactions

Must design and maintain compensating logic.



Eventual Consistency

Data may be temporarily inconsistent.



Harder to Monitor & Debug

Failures can be harder to trace across services.



Requires Idempotent Operations

Operations must be safe to retry.

Practice objective

- ✓ Order → Stock → Payment → Delivery
- ✓ Connect to Kafka Event Chain
 - OrderCreated, StockReserved, PaymentApproved, DeliveryStarted
- ✓ In case of failure, complementary event issued
 - StockReserveFailed
 - PaymentFailed
 - OrderCancelled
 - StockReleased

Practice conditions

- ✓ Each service has its own DB/status
- ✓ Do not call other services directly
- ✓ Issue status change results as events
- ✓ The following services subscribe to the event
- ✓ Issue a complementary event if it fails

Tools

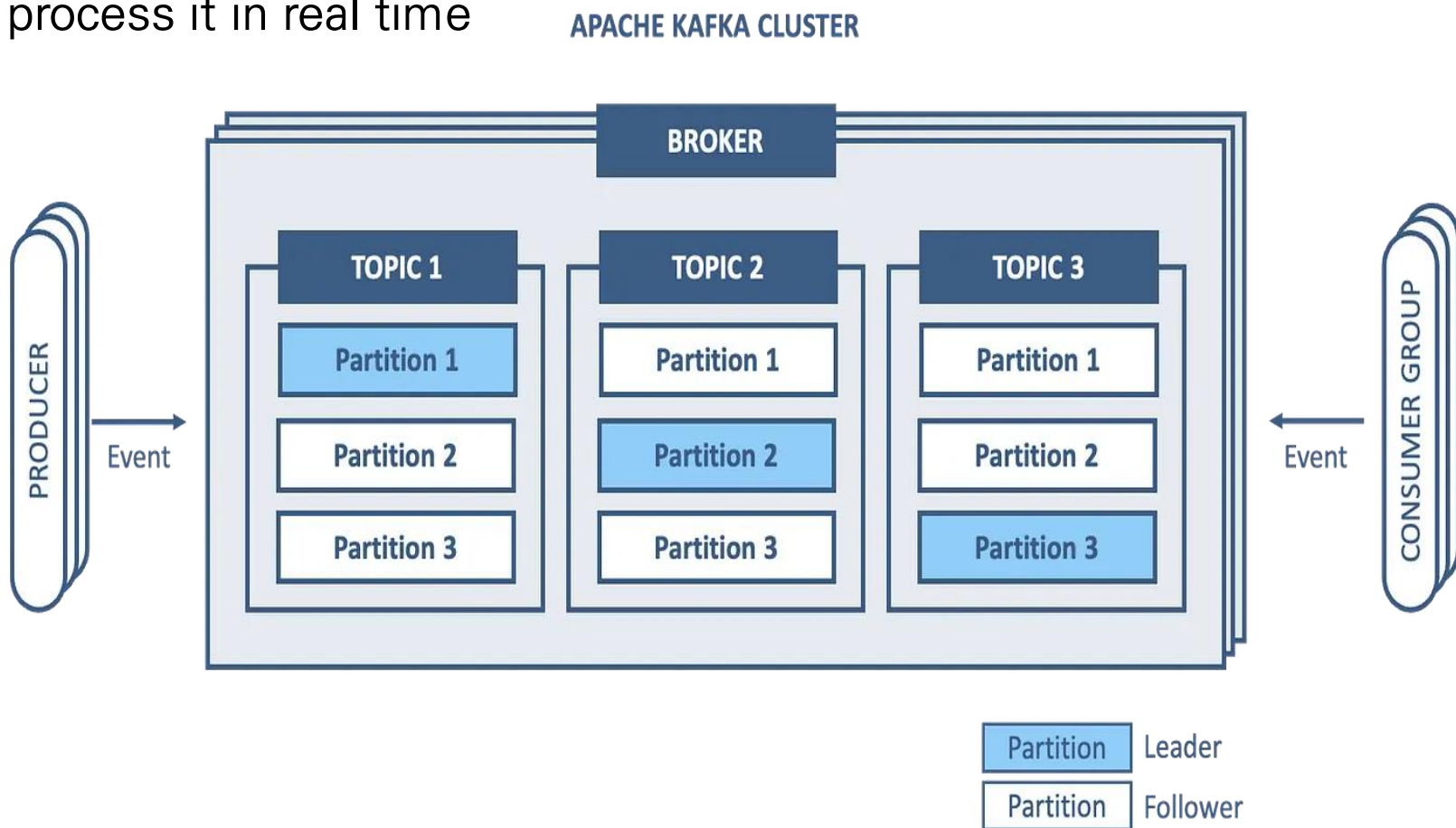
- ✓ Docker Desktop Kubernetes
- ✓ Strimzi Operator
- ✓ Kafka single-node cluster
- ✓ Kafka UI Choose one
 - Kafka UI
 - Or AKHQ

Services

- ✓ order-service
- ✓ stock-service
- ✓ payment-service
- ✓ delivery-service

Kafka concept

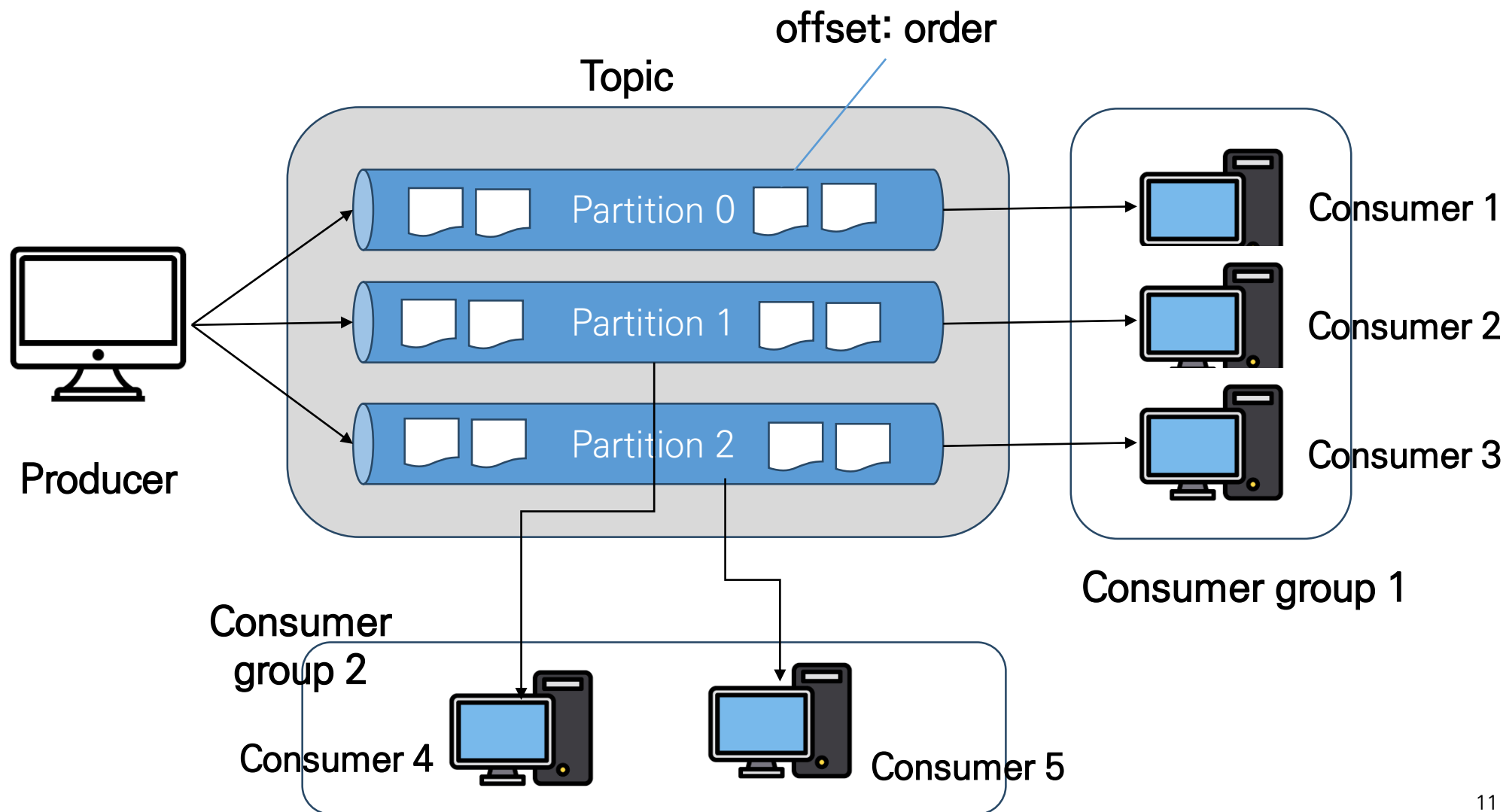
- ✓ Kafka stores data in the form of events,
- ✓ Distributed event processing system that allows multiple systems to read and process it in real time



Kafka concept

- ✓ Topic : "Logical channel" where the data goes in
 - data e.g.: order-events, payment-events
- ✓ Producer : Who sends data to Kafka
 - order-service → order-events
- ✓ Consumer : Data Readers
 - stock-service ← order-events
- ✓ Partition : Unit for distributed storage of internal data of a topic
- ✓ Group: A logical set of consumers that make up a service

Kafka concept



Kafka concept

- ✓ The number of partitions is determined by the system operator and must be greater than the number of consumer objects (pod)
- ✓ Partitions are determined by the key (or ID) that separates one event flow

e.g : OrderCreated → orderId=1001

PaymentCompleted → orderId=1001

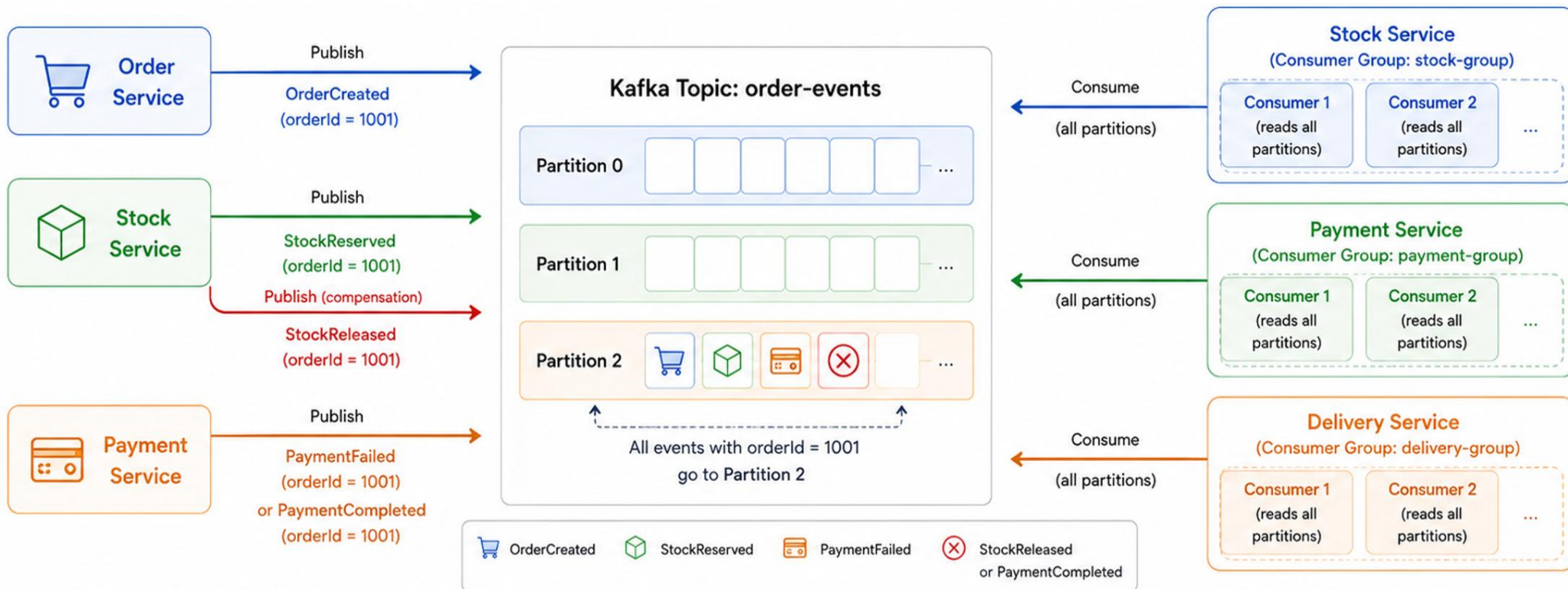
DeliveryStarted → orderId=1001

OrderCompleted → orderId=1001

} Using the same
partition

- ✓ Important to ensure the order of events

Kafka concept

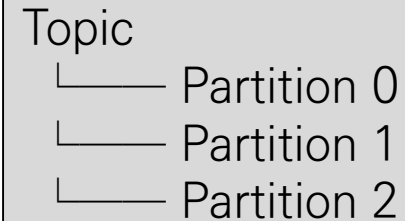


Kafka concept

- ✓ The real core of Kafka is log structure
 - [0] OrderCreated
 - [1] OrderCreated
 - [2] OrderCancelled
 - [3] OrderCreated
- ✓ Characteristics:
 - Data is append-only (continues to be added later)
 - Do not delete/modify (by default)
 - Keep the order

Kafka Internal Structure

- ✓ Partition
 - Parallelism + Scalability Core
 - Each partition ensures order
 - Different parts are parallelized
- ✓ Message saving
 - [0] OrderCreated
 - [1] OrderCreated
 - [2] OrderCancelled
 - Each message has offset (order)



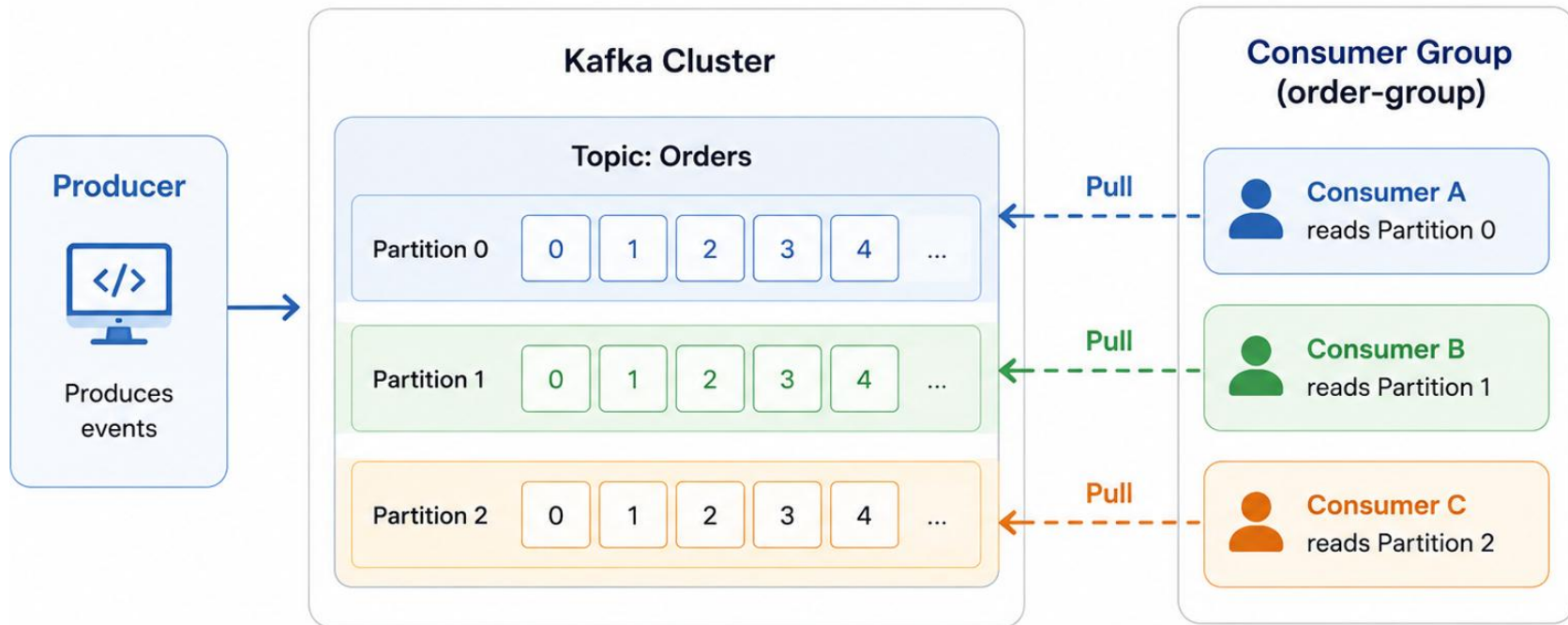
```
graph TD; Topic --> P0[Partition 0]; Topic --> P1[Partition 1]; Topic --> P2[Partition 2];
```

Topic

- └── Partition 0
- └── Partition 1
- └── Partition 2

How Consumer Works

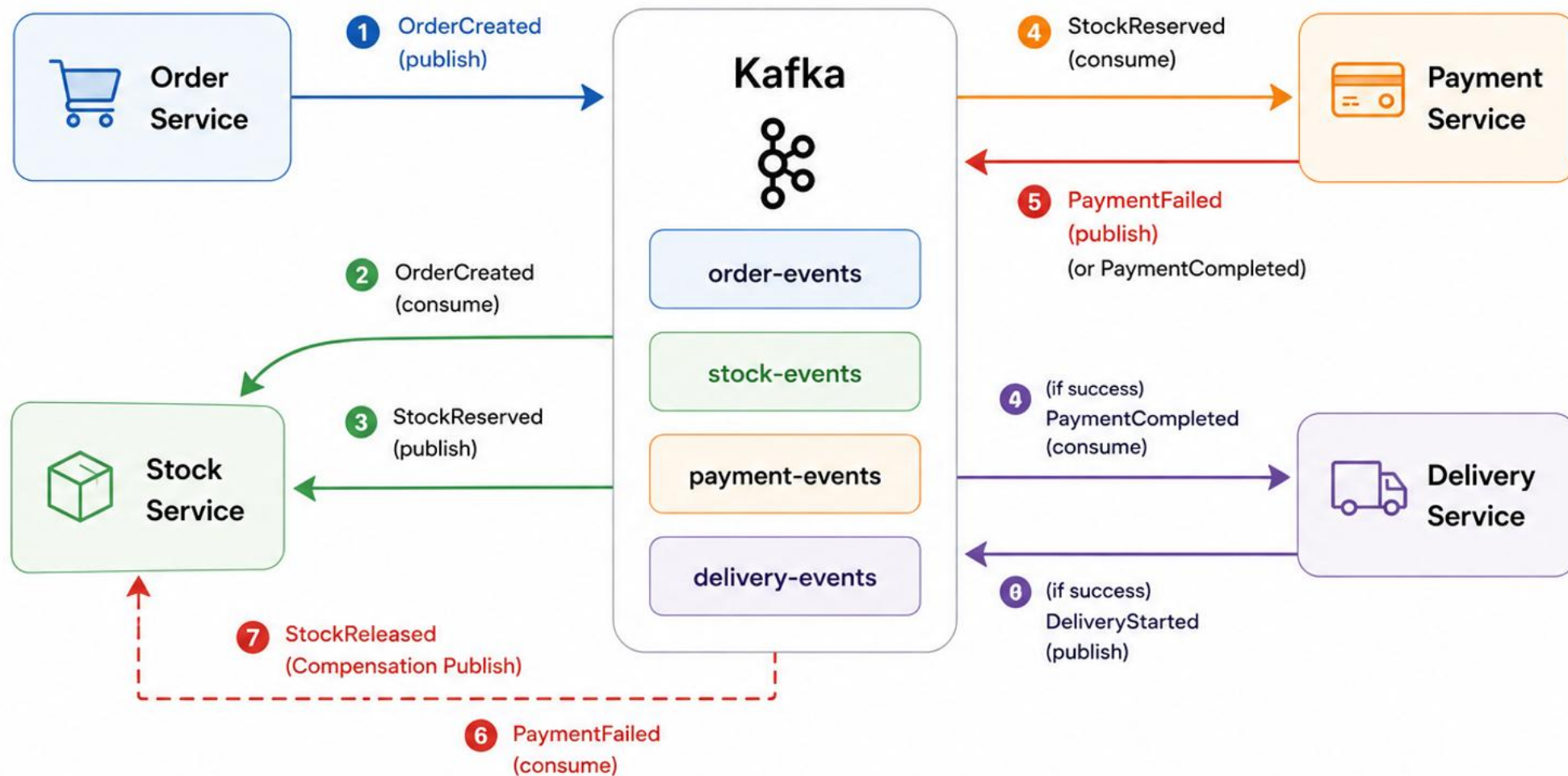
- ✓ Kafka is not push but pull method (event remains after processing)
 - Reprocessable
 - Easy to recover from a failure



The importance of Kafka

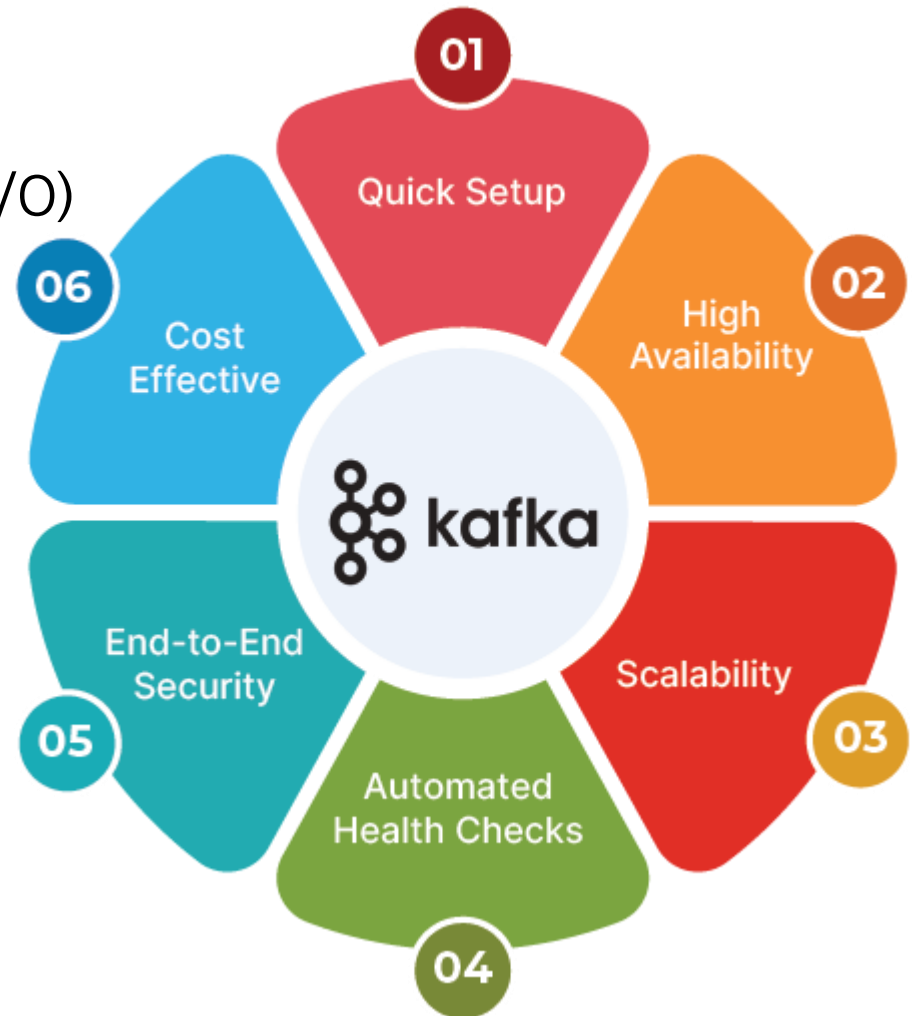
- ✓ Loose coupling
- ✓ Asynchronous processing
- ✓ Scalability
- ✓ Disability isolation

Kafka in SAGA



Key Features of Kafka

- ✓ High performance :
 - Disk-based but fast (Sequence I/O)
- ✓ Scalability
 - partition add scale-out
- ✓ durability
 - replication Support
- ✓ Reprocessable
- ✓ Multiple consumption available



Compare Kafka and NATS

category	Apache Kafka	NATS
basic purpose	Streaming Large Events / Storage Logs / Data Pipeline	Ultra-light messaging / real-time communications / microservices connectivity
Storage Method	Disk-based long-term storage	Memory Centric (Saveable with JetStream)
Preserving Messages	Strong (Retention Policy)	Default Short, Extend to JetStream
Processing speed	High	Very High (Ultra Low Latency)

Installing Kafka: Using Strimzi

Strimzi : Kafka only operator that declares Kafka as a Kubernetes resource (CRD) and automatically creates, operates, and restores it

- ✓ Kubernetes Friendly
- ✓ Operator-based
- ✓ Manage Kafka, Topic, and Users in CRD

Kafka Installation Procedure (Docker Desktop Kubernetes)

- ✓ `kubectl create namespace kafka`
- ✓ `kubectl apply -f https://strimzi.io/install/latest?namespace=kafka -n kafka`
- ✓ `kubectl get pod -n kafka --watch`
- ✓ `kubectl apply -f https://strimzi.io/examples/latest/kafka/kafka-single-node.yaml -n kafka`
- ✓ `kubectl wait kafka/my-cluster --for=condition=Ready --timeout=300s -n kafka`

Kafka UI install: provectuslabs/kafka-ui:latest

The screenshot displays the Kafka UI interface for a cluster named 'my-cluster'. The left sidebar shows the navigation menu with 'Topics' selected. The main panel shows the 'order-events' topic with the 'Messages' tab active. The interface includes filters for 'Seek Type' (Offset), 'Partitions' (All items are selected.), 'Key Serde' (String), and 'Value Serde' (String). A search bar and '+ Add Filters' button are also present. Below the filters, a table lists 7 messages with columns for Offset, Partition, Timestamp, Key, and Preview.

	Offset	Partition	Timestamp	Key	Preview
+	0	0	2026. 4. 3. 0시 3분 43초	0efd2cab-3ca4-409e-b254-6a955542de98	
+	1	0	2026. 4. 3. 0시 13분 13초	6fe236fb-f7c5-4687-8cbc-7773353a4054	
+	2	0	2026. 4. 3. 15시 18분 30초	03028395-6c30-4fb2-9506-39a1e6b65118	
+	3	0	2026. 4. 3. 15시 19분 36초	88a50740-13a6-4e08-8419-f15b96f00168	
+	4	0	2026. 4. 3. 15시 19분 58초	28d80ea0-ff0b-48d1-94f6-c795f3806eaf	
+	5	0	2026. 4. 3. 15시 20분 3초	b9968047-087b-426a-8504-edec7c3709f0	
+	6	0	2026. 4. 3. 15시 20분 11초	2661df23-9e15-45fa-a180-8be137a2191f	

Kafka basic practice

- ✓ Example of simple event creation (producer) and consumption (consumer)
- ✓ Topic serves as an order-event
- ✓ The producer should be the event serialize and the volume should be the event desialize
- ✓ Jackson library dependencies must be included

role	Description
Producer	Issue of events
Kafka	Save Event
Consumer	Event Processing

Kafka basic practice

- ✓ No KafkaConnector is required to send and receive regular Kafka events

Order Service
└─ KafkaTemplate → Kafka Topic

Payment Service
└─ @KafkaListener ← Kafka Topic

```
spring:
  kafka:
    bootstrap-servers: my-cluster-kafka-bootstrap.kafka:9092
```

Kafka basic practice

- ✓ The CDC is structured to read DB change logs and turn them into Kafka events, rather than applications creating events themselves
- ✓ KafkaConnector is intended to manage Debezium Connector settings with Kubernetes resources in Strimzi

Sortation	General Kafka Event	CDC
Event creator	application code	Debezium Connector
KafkaConnector need	Unnecessary	Necessity
Data Source	Service logic	DB Change Log
Use Examples	Issue Order Creation	order table INSERT detection
Implementation Method	KafkaTemplate, @KafkaListener	Kafka Connect + Debezium

Kafka basic practice

← → ↺ 🏠 ⓘ localhost:30080/index.html ☆ 👤 ⋮

🍷 | 🇸🇰 쿠팡!



Kafka Event Monitor

Real-time Producer & Consumer Event Tracking

Create Order Event

Product ID Quantity Amount

Send Event Clear All

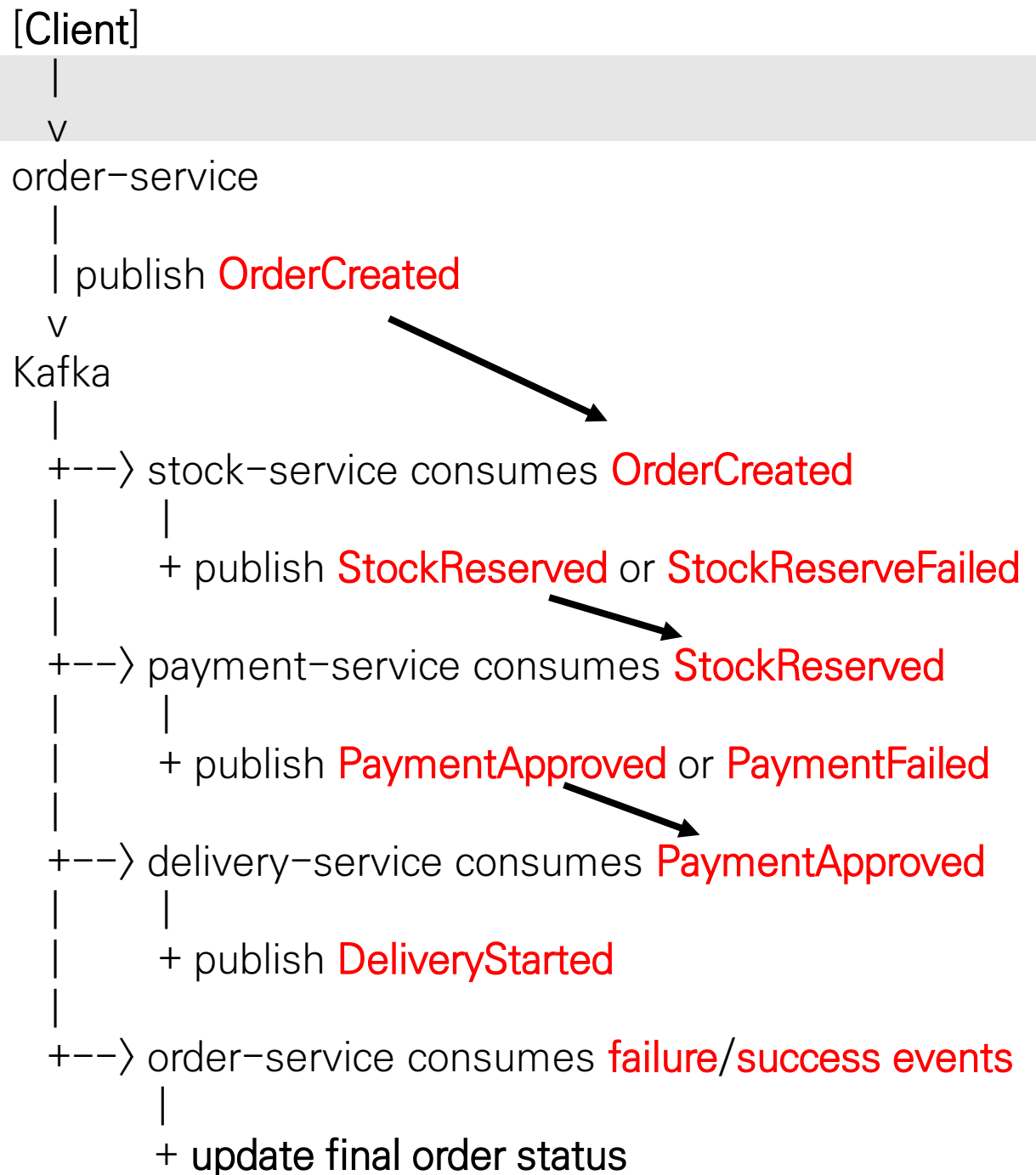
● Producer Events 6

Event Type	Order ID	Product ID	Qty	Amount
OrderCreated	9f382691-9a09-453d...	PROD-001	1	100
OrderCreated	9e2540ba-50d1-4f41...	PROD-001	1	100

● Consumer Events 6

Event Type	Order ID	Product ID	Qty	Amount
OrderCreated	9f382691-9a09-453d...	PROD-001	1	100
OrderCreated	9e2540ba-50d1-4f41...	PROD-001	1	100

Practical Event Flow



Order Status

- ✓ CREATED
- ✓ STOCK_CONFIRMED
- ✓ PAYMENT_CONFIRMED
- ✓ COMPLETED
- ✓ CANCELLED

Practice Scenario 1: Out of stock

1. Create Order
2. Issue Order Creation
3. Inventory Service Verification Quantity
4. Failure to prepare inventory if it is out of stock
5. Your order service has been canceled

Practice Scenario 2: Payment Failed

1. Create Order
2. Successfully Stocked → StockReserved
3. Payment failed → PaymentFailed
4. StockReleased because the inventory service received PaymentFailed
5. Order service is final CANCELED

Practice Scenario 3: All Successful

1. OrderCreated
2. StockReserved
3. PaymentApproved
4. DeliveryStarted
5. Order status COMPLETED

Topic Design

- ✓ order-events
- ✓ stock-events
- ✓ payment-events

Separate topics by service

- ✓ order-service → order-events
- ✓ stock-service → stock-events
- ✓ payment-service → payment-events

Example of Event Message Structure

```
{  
  "eventId": "uuid",  
  "eventType": "OrderCreated",  
  "sagaId": "uuid",  
  "orderId": "1001",  
  "timestamp": "2026-03-31T18:00:00",  
  "payload": {  
    "productId": "P001",  
    "quantity": 2,  
    "amount": 30000  
  }  
}
```

SAGA Implementation Method

- ✓ Choreography
 - Each service sees the event and takes the next action
 - No central coordinator
- ✓ Orchestration
 - A central orchestra conducts each service

Role by Service

✓ order-service

- Provide Order Generation API
- Save Order
- OrderCreated issue
- Consume success/failure events
- Update Final Status

✓ payment-service

- Stock Reserve Subscription
- Accept payment in successful/failure simulation
- Fail if payment fails

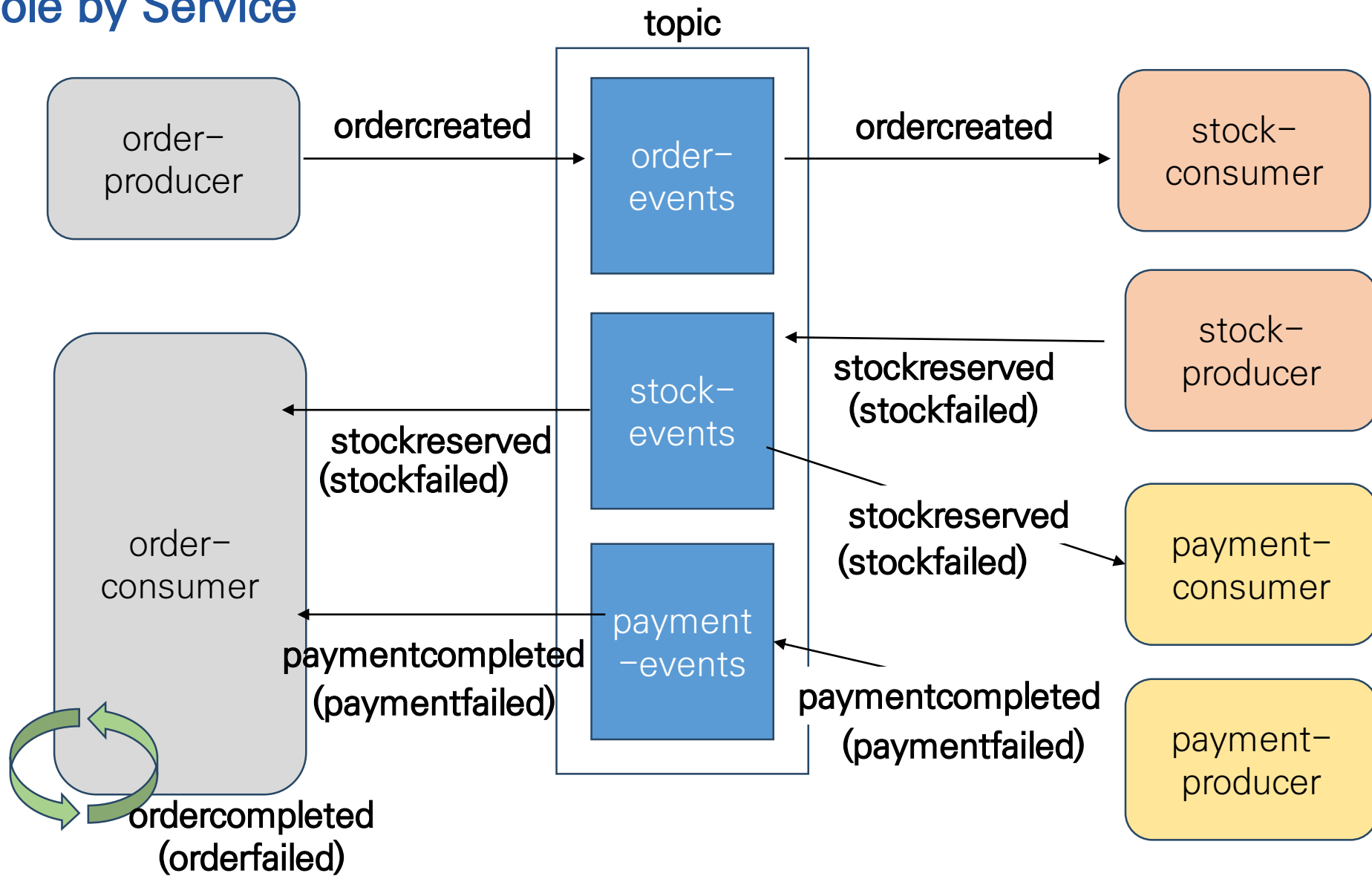
✓ stock-service

- OrderCreated subscription
- Stock Reserved in case of successful inventory check and deduction
- When StockReserveFailed
- StockReleased if payment fails

✓ delivery-service

- Payment Authorization Subscription
- Create Delivery
- Delivery Start Issuance

Role by Service



How to Simulate Failure

- ✓ Lead to failure with request values
 - Example: Inventory failure if product quantity is greater than 100
 - Payment failed if the amount is more than 50000
 - Delivery failure with specific product code

Practice Roadmap

- ✓ Step 1: Installing Kafka
 - Install Strimzi
 - Kafka single-node create
 - Create Topic
 - Producer/consumer test
- ✓ Step 2 : order-service + stock-service
 - Create Order
 - Issue Order Event
 - inventory service consumption
 - Reply success/failure events

Practice Roadmap

- ✓ Step 3: Add payment-service
 - Payments processed after StockReserved
 - Reward event in case of failure
- ✓ Step 4: Add observation points
 - Kafka UI
 - Logs
 - Status Table
 - tracking sagald

Practice 1: Order Success Case

← → ↺ 🏠 ⓘ localhost:30300/index.html 🔍 ☆ 📁 👤

🍷 새 탭

8 Container Orches...

Committee ISO/TC...

ISO/TC 307 - Block...

쿠버네티스 시작하...

클라우드스토어 씨...

Home Page - Clou...

>> 📁 모든 !

Order Saga

주문 정보를 웹에서 등록하면 MySQL에 저장됩니다.

상품 ID

p100

재고 확인

현재 재고: 4개

수량

1

금액

1000

주문 생성

이벤트 로그

Kafka로 들어온 이벤트/주문 서비스 이벤트를 표시합니다. (5초 자동 갱신)

시간	소스	이벤트	주문ID	비고
2026. 4. 4. 오후 1:29:34	STOCK_SERVICE	StockReleased	303a6eac-1246-4d55-88a2-8bcb3e6232ff	
2026. 4. 4. 오후 1:29:34	ORDER_SERVICE	OrderCancelled	303a6eac-1246-4d55-88a2-8bcb3e6232ff	재고 부족
2026. 4. 4. 오후 1:29:34	ORDER_SERVICE	OrderCancelled	303a6eac-1246-4d55-88a2-8bcb3e6232ff	재고 부족
2026. 4. 4. 오후 1:29:34	STOCK_SERVICE	StockFailed	303a6eac-1246-4d55-88a2-8bcb3e6232ff	재고 부족
2026. 4. 4. 오후 1:29:34	STOCK_SERVICE	StockFailed	303a6eac-1246-4d55-88a2-8bcb3e6232ff	재고 부족
2026. 4. 4. 오후 1:29:34	ORDER_SERVICE	OrderCreated	303a6eac-1246-4d55-88a2-8bcb3e6232ff	

Practice 2: Out of stock case

← → ↺ 🏠

localhost:30300/index.html

🔍 ☆ 📌

🍷 새 탭

8 Container Orches...

Committee ISO/TC...

ISO/TC 307 - Block...

쿠버네티스 시작하...

클라우드스토어 씨...

Home Page - Clou...

» 📁 모든

Order Saga

주문 정보를 웹에서 등록하면 MySQL에 저장됩니다.

상품 ID

p100

재고 확인

현재 재고: 4개

수량

5

금액

1000

주문 생성

이벤트 로그

Kafka로 들어온 이벤트/주문 서비스 이벤트를 표시합니다. (5초 자동 갱신)

시간	소스	이벤트	주문ID	비고
2026. 4. 4. 오후 1:29:34	STOCK_SERVICE	StockReleased	303a6eac-1246-4d55-88a2-8bcb3e6232ff	
2026. 4. 4. 오후 1:29:34	ORDER_SERVICE	OrderCancelled	303a6eac-1246-4d55-88a2-8bcb3e6232ff	재고 부족
2026. 4. 4. 오후 1:29:34	ORDER_SERVICE	OrderCancelled	303a6eac-1246-4d55-88a2-8bcb3e6232ff	재고 부족
2026. 4. 4. 오후 1:29:34	STOCK_SERVICE	StockFailed	303a6eac-1246-4d55-88a2-8bcb3e6232ff	재고 부족
2026. 4. 4. 오후 1:29:34	STOCK_SERVICE	StockFailed	303a6eac-1246-4d55-88a2-8bcb3e6232ff	재고 부족
2026. 4. 4. 오후	ORDER_SERVICE	OrderCreated	303a6eac-1246-4d55-88a2-	

Practice 3: Payment failure case → failure if the amount is greater than 50000



Order Saga

주문 정보를 웹에서 등록하면 MySQL에 저장됩니다.

상품 ID

p100

재고 확인

현재 재고: 4개

수량

1

금액

60000

주문 생성

주문 생성 완료: f387aa1e-2db3-4a90-9a3f-bdcfa56ef9fc

이벤트 로그

Kafka로 들어온 이벤트/주문 서비스 이벤트를 표시합니다. (5초 자동 갱신)

시간	소스	이벤트	주문ID	비고
2026. 4. 4. 오후 1:41:36	STOCK_SERVICE	StockReleased	f387aa1e-2db3-4a90-9a3f-bdcfa56ef9fc	
2026. 4. 4. 오후 1:41:36	ORDER_SERVICE	OrderCancelled	f387aa1e-2db3-4a90-9a3f-bdcfa56ef9fc	결제 승인 실패
2026. 4. 4. 오후 1:41:36	ORDER_SERVICE	OrderCancelled	f387aa1e-2db3-4a90-9a3f-bdcfa56ef9fc	결제 승인 실패
2026. 4. 4. 오후 1:41:36	ORDER_SERVICE	StockReserved	f387aa1e-2db3-4a90-9a3f-bdcfa56ef9fc	
2026. 4. 4. 오후 1:41:36	PAYMENT_SERVICE	PaymentFailed	f387aa1e-2db3-4a90-9a3f-bdcfa56ef9fc	결제 승인 실패
2026. 4. 4. 오후	STOCK_SERVICE	StockReserved	f387aa1e-2db3-4a90-9a3f-	

Exercise 4: Inventory adjustment

→ ↺ 🏠 ⓘ localhost:30302/index.html 🔍 ☆ 📁 | 👤

🖱 새 탭 🍌 8 Container Orches... 📄 Committee ISO/TC... 📄 ISO/TC 307 - Block... 📄 쿠버네티스 시작하... 🌐 클라우드스토어 씨... 📄 Home Page - Clou... >> | 📁 모든

Stock Saga Dashboard

재고와 Saga 이벤트 상태를 실시간으로 확인합니다. 새로고침

재고 현황

상품 수: 4 마지막 변경: 2026-04-04T13:41:36.531102

상품ID	수량	업데이트
P100	4	2026-04-04T13:41:36.531102
P200	1	2026-04-04T03:25:11.365849
p300	6	2026-04-04T13:29:34.348531
p400	21	2026-04-04T13:14:26.64778

재고 수량 등록/수정

저장

주문(OrderCreated) 이벤트가 오면 해당 상품 재고가 자동 차감됩니다.

이벤트 상태 로그 (최신 200건)

로그 수: 127 최신 이벤트: 2026-04-04T13:41:36.544962

Differences from **synchronization call method**

✓ REST Chain:

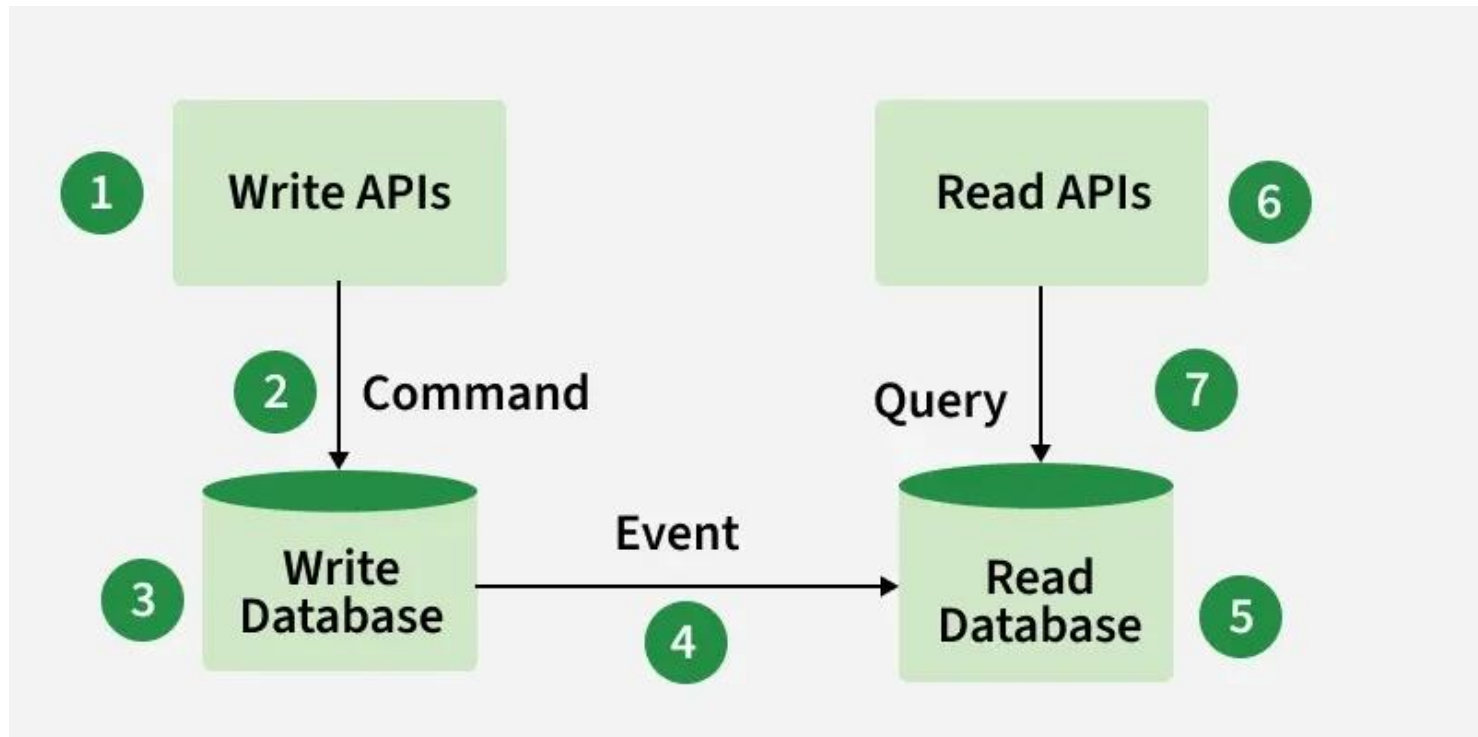
- Front service waits for back service response
- Ease of Disability Transmission
- High degree of coupling

✓ Event-based:

- Asynchronous processing after issue
- a loose bond
- Final consistency
- Compensation transaction required

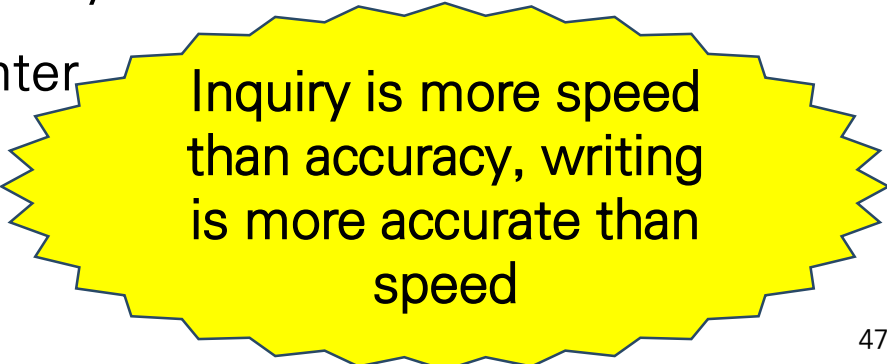
CQRS: Command Query Responsibility Segregation

- ✓ Architecture patterns separating write (command) and read (Query)
- ✓ Command (write) → Status change → Event occurs
- ✓ Query → Inquiry-only model → Quick response



CQRS Reasons

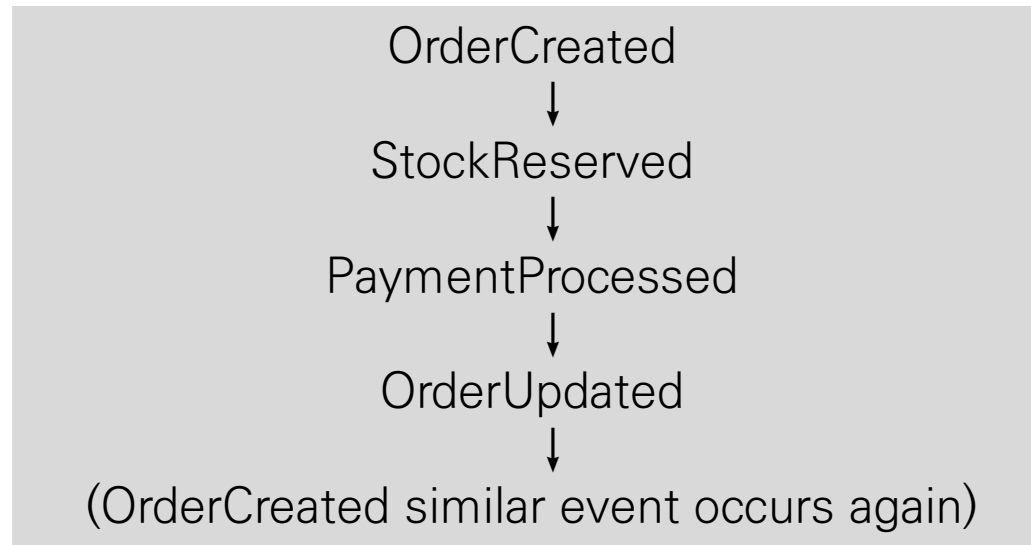
- ✓ Performance
 - Separating inquiry from write, inquiry is fast (cash, read replica)
Writing is safe (transaction, event)
- ✓ Separation of complexity
 - Write: Complex (Inventory, Payment, Compensation)
 - Inquiry: Simple (Show Value)
- ✓ Scalability
 - Lookup Traffic High → Scale Separately
 - Less Write Traffic → Reliability Center



Inquiry is more speed
than accuracy, writing
is more accurate than
speed

Note: Event Loop

- ✓ If an event is looped when an event is designed incorrectly, the message will explode
- ✓ Example:



Note: Event Loop

Cause	Description
Event reissue condition error	Issue the same event for each status change
Saga compensation logic error	Failure → Compensation → Repeat Retry
redundant consumption	Lack of idempotency
Cross-Triggering Multiple Services	$A \rightarrow B \rightarrow C \rightarrow A$
DLQ reprocessing error	Repeat Failure Events

Event Loop Prevention Measures

- ✓ Clarify event type
 - E.g : OrderCreated
OrderConfirmed
OrderCancelled
 - Do not reissue the same event
- ✓ Separate Topic Roles
 - commands-topic
 - events-topic
 - compensation-topic

Event Loop Prevention Measures

- ✓ Idempotency
 - Save eventId
 - Prevent redundancy
 - ✓ Saga Health Management
 - tracking sagaId
 - Save Step-by-Step Status
 - ✓ TTL / Retry Restrictions
 - Maximum Retries
 - Utilize DLQs
- Time to Live (TTL): Maximum survival time when messages, data, caches, and resources are valid
 - Dead Letter Queue / Dead Letter Topic (DLQ): A space for separately isolating messages that have failed normal processing

Success Scenario

1. Create Order
2. Order Service → OrderCreated
3. Orchestrator → Stock Service ReserveStockCommand
4. Stock Success → StockReserved
5. Orchestrator → Payment Service ProcessPaymentCommand
6. Payment Success → PaymentCompleted
7. Orchestrator → Order Service에 CompleteOrderCommand
8. Order completed

Failure Scenario 1: Out of stock

1. Create Order
2. Stock failed → StockFailed
3. Orchestrator → FailOrderCommand to Order Service
4. Order failed

Failure Scenario 2: Payment failure

1. Create Order
2. Stock Success
3. Payment failed → PaymentFailed
4. Orchestrator → ReleaseStockCommand to Stock Service
5. Restoring Inventory
6. Orchestrator → FailOrderCommand to Order Service
7. Order failed

Service

- ✓ order-service
- ✓ stock-service
- ✓ payment-service
- ✓ orchestrator-service

Topic

- ✓ order-events
- ✓ stock-commands
- ✓ stock-events
- ✓ payment-commands
- ✓ payment-events
- ✓ order-commands

SAGA Orchestration Practice

